

Progetto di Apprendimento Automatico e Profondo

Object Detection e Sentiment Analysis: da YOLO ai Transformer

Matteo Barbieri

Apprendimento Profondo

24 settembre 2026

- 1 Introduzione al deep learning
- 2 Addestrare una rete profonda
- 3 Object Detection (YOLO)
- 4 Sentiment Analysis (Transformer)
- 5 Conclusioni

- 1 Introduzione al deep learning
- 2 Addestrare una rete profonda
- 3 Object Detection (YOLO)
- 4 Sentiment Analysis (Transformer)
- 5 Conclusioni

Che cos'è il deep learning

Studio e applicazione di metodi basati su **reti neurali** per problemi ad alto impatto: visione, linguaggio, parlato.

Machine learning classico

dati $\rightarrow$ **feature definite a mano** (selezione, costruzione, trasformazione) $\rightarrow$ Learning $\rightarrow f(X)$

La definizione delle feature è manuale: è il passo *fondamentale* della pipeline.

Deep learning

dati $\rightarrow$ trasformazioni **apprese** $z_1 \rightarrow z_2 \rightarrow \dots \rightarrow z_h$
 $\rightarrow$ Classifier $\rightarrow f(X)$

Rappresentazioni intermedie via via più astratte, es.
pixel $\rightarrow$ edge $\rightarrow$ motif $\rightarrow$ parti $\rightarrow$ oggetti
(*representational learning*).

- Funzioni complesse calcolate **annidando** (*nesting*) funzioni semplici: una sequenza di layer sempre più **astratti**.
- Approcci principali: reti **feedforward** profonde, reti **convoluzionali** (CNN), reti **ricorrenti**, **Transformer**, ecc.

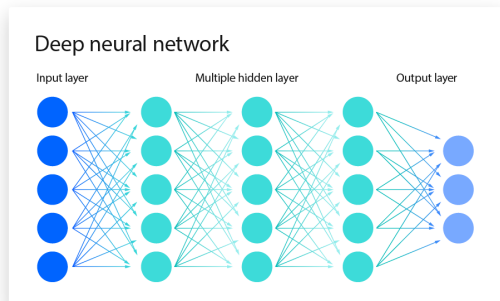
Modello del neurone

Combinazione lineare degli input seguita da una **funzione di attivazione** φ :

$$y = \varphi\left(\sum_i \theta_i x_i\right) = h_{\theta}(x)$$

- $\theta_1, \dots, \theta_n = \mathbf{pesi}$; $\theta_0 = \mathbf{bias}$, con la *bias unit* $x_0 = 1$.
- Con $\varphi = \text{logistica}$ il neurone è la **regressione logistica**.
- Senza φ il neurone è solo un **prodotto scalare**: è φ a dare la **non linearità**.

- I neuroni si organizzano in **layer**: input, layer **nascosti**, output.
- Rete **deep-L**: L = numero di layer nascosti + output.
- $\theta_{ik}^{(l)}$ = peso dall'unità k del layer l all'unità i del layer $l + 1$; $\Theta^{(l)}$ ne è la matrice.



Forward propagation

L'output di un layer è l'input del successivo:

$$a_i^{(1)} = x_i, \quad a_i^{(l)} = \varphi\left(\sum_k \theta_{ik}^{(l-1)} a_k^{(l-1)}\right) \quad 1 < l \leq L$$

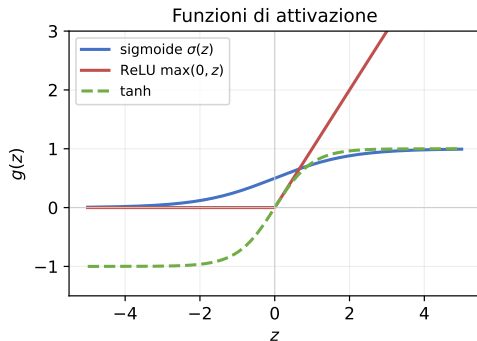
- In forma vettoriale: $\mathbf{a}^{(l)} = \varphi(\Theta^{(l-1)} \mathbf{a}^{(l-1)})$, con la bias unit $a_0^{(l-1)} = 1$.
- L'output della rete è $h_{\Theta}(\mathbf{x}) = \mathbf{a}^{(L)}$.
- Ogni passo è un **prodotto matrice-vettore** seguito da φ applicata elemento per elemento.

Perché funzioni non lineari?

Con attivazioni **lineari**, la composizione di più layer equivale a un **singolo layer lineare**. Un modello puramente lineare può solo tracciare iperpiani rigidi nello spazio dei dati, risultando inadeguato.

Funzioni di attivazione

	$\varphi(z)$	$\varphi'(z)$
Logistica	$\frac{1}{1 + e^{-z}}$	$\varphi(z)(1 - \varphi(z))$
tanh	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	$1 - \varphi(z)^2$
ReLU	$\max(0, z)$	0 se $z < 0$ 1 se $z \geq 0$
Leaky ReLU	$\max(\alpha z, z)$	α se $z < 0$ 1 se $z \geq 0$
ELU	$\alpha(e^z - 1)$ se $z < 0$, z altrimenti	$\varphi(z) + \alpha$



- **Layer nascosti:** logistica fino agli anni '90, poi tanh, oggi **ReLU**.
- **Output:** lineare per la regressione, logistica per la classificazione binaria, **softmax** per la multiclasse.

Pro e contro delle funzioni di attivazione

Logistica (Sigmoidale) e tanh

- + **Derivabili** ovunque.
- + Logistica in $[0, 1]$ (ottima per probabilità); tanh è centrata in 0.
- **Saturazione:** per $|z|$ grande il gradiente tende a 0 (vanishing gradient).
- Logistica sempre positiva (gli aggiornamenti dei pesi oscillano).
- **Nessuna sparsità** (mai esattamente 0) e costo computazionale elevato (exp).

ReLU

- + **Computazione efficiente** (soluzione veloce e derivata immediata).
- + **Non satura** per $z > 0$ (evita il vanishing gradient).
- + Genera **sparsità**: produce zeri effettivi disattivando neuroni non necessari.
- **Dying ReLU:** se un neurone muore (sempre 0), non impara più.

Loss e Funzione di Costo

Loss $\ell(\hat{y}, y)$: errore sul singolo esempio.

Costo $J(\Theta)$: media dell'errore su m esempi: $J(\Theta) = \frac{1}{m} \sum_{i=1}^m \ell(h_{\Theta}(x^{(i)}), y^{(i)})$

Loss	Ambito	Formula
MSE	Regressione ($y \in \mathbb{R}$)	$\frac{1}{2}(\hat{y} - y)^2$
Cross-Entropy	Class. Binaria ($y \in \{0, 1\}$)	$-y \log \hat{y} - (1 - y) \log(1 - \hat{y})$
Softmax + CE	Class. Multiclasse	$-\sum_i y_i \log \hat{y}_i$

Esempio: Object Detector (es. YOLO)

Combinazione di più loss in un unico modello:

- **MSE** per la posizione della bounding box.
- **Cross-Entropy Binaria** per la presenza dell'oggetto.
- **Softmax** per la classe dell'oggetto.

- 1 Introduzione al deep learning
- 2 Addestrare una rete profonda**
- 3 Object Detection (YOLO)
- 4 Sentiment Analysis (Transformer)
- 5 Conclusioni

Regola di aggiornamento

Ogni peso si sposta nella direzione **opposta al gradiente**:

$$w_{ij} := w_{ij} - \eta \frac{\partial E}{\partial w_{ij}}$$

- **Derivata parziale** $\frac{\partial E}{\partial w_{ij}}$: indica quanto varia l'errore E modificando il solo peso w_{ij} (con gli altri costanti).
- **Learning rate** (η): regola l'ampiezza del passo. Se troppo grande sbalza oltre il minimo, se troppo piccolo la convergenza è lentissima.
- **Backpropagation**: calcola le derivate parziali per tutti i pesi.
- **Ottimizzatori avanzati**: Mini-batch, Momentum, RMSprop, Adam.

Backpropagation

Si propaga l'errore **all'indietro**, dal layer di output verso l'input, con la *regola della catena*. Per ogni unità si calcola un errore $\delta_i^{(l)}$:

$$\delta_i^{(L)} = \ell'(h_{\Theta}(x^{(i)}))$$

errore sull'output

$$\delta_i^{(l)} = \left(\sum_k \delta_k^{(l+1)} \theta_{ki}^{(l)} \right) \varphi'(a_i^{(l)})$$

errore riportato indietro

$$\frac{\partial J(\Theta)}{\partial \theta_{ij}^{(l)}} = a_j^{(l)} \delta_i^{(l+1)}$$

gradiente di ogni peso

- L'errore di un'unità nascosta usa la **derivata della sua funzione di attivazione**.
- Un passo di training = forward propagation + backpropagation + aggiornamento dei pesi.

Gradienti instabili: vanishing ed exploding

In una catena di unità con bias b_i , pesi w_i , attivazione σ e costo C , il gradiente di un layer iniziale è un **prodotto**:

$$\frac{\partial C}{\partial b_1} = \sigma'(z_1) w_2 \sigma'(z_2) w_3 \sigma'(z_3) w_4 \sigma'(z_4) \frac{\partial C}{\partial a_4}$$

Vanishing

Se ogni prodotto $w_i \sigma'(z_i) < 1$ il gradiente **svanisce**. Con la logistica $0 \leq \sigma'(z) \leq 0.25$ e pesi da $\mathcal{N}(0, 1)$ succede facilmente.

Exploding

Se ogni prodotto $w_i \sigma'(z_i) > 1$, per esempio con pesi inizializzati grandi, il gradiente **esplode** verso i primi layer.

In entrambi i casi i layer imparano a **velocità diverse**: i primi layer, con i prodotti più lunghi, sono i più colpiti.

Inizializzazione dei pesi

Con input standardizzati e pesi da $\mathcal{N}(0, 1)$, la somma $z = \theta_1 x_1 + \dots + \theta_n x_n$ ha media 0 e **varianza** n (deviazione standard $\sqrt{n}$). Per n grande z tende a esplodere o svanire.

Soluzione: riscaldare i pesi di ogni layer

Si mantiene deviazione standard 1 in ogni layer moltiplicando per un fattore che dipende da $n^{(l-1)}$, il numero di input del layer:

$$\theta_{ij}^{(l)} = \mathcal{N}(0, 1) \cdot \sqrt{1/n^{(l-1)}}$$

attivazione lineare

$$\theta_{ij}^{(l)} = \mathcal{N}(0, 1) \cdot \sqrt{2/n^{(l-1)}}$$

ReLU (inizializzazione di **He**)

$$\theta_{ij}^{(l)} = \mathcal{N}(0, 1) \cdot \sqrt{2/(n^{(l-1)} + n^{(l)})}$$

tanh (inizializzazione di **Xavier**)

Sigmoidi con inizializzazione sbagliata $\Rightarrow$ gradienti instabili; **ReLU** + **He** è la combinazione standard.

Overfitting e regolarizzazione L2

La regolarizzazione riduce l'**overfitting**, problema centrale nel deep learning, dove le reti hanno moltissimi parametri.

Costo regolarizzato

$$J(\Theta) = \frac{1}{m} \sum_{i=1}^m \ell(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \sum_{l=1}^L \|\Theta^{(l)}\|^2$$

- $\lambda \uparrow \Rightarrow \theta_{ij}^{(l)} \downarrow$: molti pesi vicini a 0, reti più semplici e sparse, meno overfitting.
- **Effetto sull'attivazione**: con pesi piccoli l'argomento z è vicino a 0, dove la funzione di attivazione è quasi **lineare**. La rete approssima solo funzioni meno complesse.

Nel progetto la regolarizzazione L2 è il *weight decay* passato all'ottimizzatore.

Dropout, data augmentation, early stopping

Dropout

Alternativa alla L2: ogni nodo viene **eliminato** con probabilità p .

- p unica, diversa per layer (più alta nei layer grandi) o solo su alcuni layer.
- Reti più semplici; nessuna unità può contare su una singola feature, quindi i pesi si distribuiscono.
- **Mai a test time.**

Data augmentation

Nuovi esempi trasformando quelli esistenti: flip, crop, zoom, rotazioni, distorsioni, variazioni di colore.

Riduce l'overfitting ed è naturale nella visione.

Early stopping

Si ferma il training quando l'errore sul validation ricomincia a salire.

Limite: ottimizza il costo e riduce la varianza *insieme*; la L2 tiene separati i due compiti (*ortogonalità*).

Nel progetto: augmentation nel detector, dropout dentro BERT, scelta del checkpoint sul validation in entrambi.

Mini-batch gradient descent

Le reti profonde si addestrano su **big data**: calcolare il gradiente su tutti gli m esempi prima di ogni aggiornamento può essere proibitivo. Si divide il training set in k mini-batch e si aggiornano i pesi dopo **ogni** mini-batch; un'**epoca** è un passaggio completo.

Dimensione	Metodo	Comportamento
m	batch GD	può essere lento
1	stochastic GD (SGD)	veloce, ma perde la vettorizzazione
intermedia	mini-batch GD	il compromesso usato in pratica

- Batch piccoli: convergenza rapida ma soluzione sub-ottima; batch grandi: convergenza lenta ma soluzione migliore.
- Regole pratiche: con $m \leq 2000$ batch GD; altrimenti batch da 64, 128, 256 o 512, che **stiano in memoria** della CPU/GPU.

Medie mobili esponenziali

Sono il componente chiave degli ottimizzatori per reti profonde:

$$v_t = \beta v_{t-1} + (1 - \beta) \theta_t = (1 - \beta) \theta_t + (1 - \beta) \beta \theta_{t-1} + (1 - \beta) \beta^2 \theta_{t-2} + \dots$$

- I termini più vecchi pesano meno, perché β^t decresce con t .
- $\frac{1}{1-\beta}$ approssima l'ampiezza della finestra: $\beta = 0.5$ circa 2 valori, $\beta = 0.9$ circa 10, $\beta = 0.98$ circa 50.

Bias correction

Partendo da $v_0 = 0$, con $\beta = 0.98$ la curva parte troppo in basso. Si corregge ogni valore:

$$v'_t = \frac{v_t}{1 - \beta^t}$$

All'inizio il fattore normalizza la somma pesata; per t grande non ha più effetto.

Momentum, RMSprop, Adam

Sia $dw_{ij} = \partial E / \partial w_{ij}$, calcolato sul mini-batch.

Momentum

Media mobile del **gradiente**:

$$\nu_{ij} = \beta \nu_{ij} + (1 - \beta) dw_{ij}$$

$$w_{ij} = w_{ij} - \eta \nu_{ij}$$

Aiuta a **uscire dai minimi locali**.
Con β grande serve η piccolo; di solito senza bias correction.

RMSprop

Media mobile del **quadrato** del gradiente:

$$\nu_{ij} = \beta \nu_{ij} + (1 - \beta) dw_{ij}^2$$

$$w_{ij} = w_{ij} - \eta \frac{dw_{ij}}{\sqrt{\nu_{ij}} + \epsilon}$$

Normalizza le fluttuazioni: i valori grandi diventano più piccoli e viceversa.

Adam (*ADaptive Momentum*)

Momentum + RMSprop con bias correction:

$$\nu_{ij} = \frac{1}{1 - \beta_1^t} (\beta_1 \nu_{ij} + (1 - \beta_1) dw_{ij})$$

$$\sigma_{ij} = \frac{1}{1 - \beta_2^t} (\beta_2 \sigma_{ij} + (1 - \beta_2) dw_{ij}^2)$$

$$w_{ij} = w_{ij} - \eta \frac{\nu_{ij}}{\sqrt{\sigma_{ij}} + \epsilon}$$

$\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$; si regola η .

Adam è l'ottimizzatore di **entrambi** gli esercizi (nell'Es. 2 nella variante AdamW).

Batch Normalization

Idea

Come per gli input, si normalizzano i **nodi interni** z_i (prima dell'attivazione), con media μ_i e varianza σ_i^2 calcolate su ogni **mini-batch**:

$$\hat{z}_i^{(k)} = \frac{z_i^{(k)} - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}}, \quad \tilde{z}_i^{(k)} = \gamma_i \hat{z}_i^{(k)} + \beta_i$$

γ_i, β_i si apprendono: normalizzare a media 0 e varianza 1 non conviene, perché lì l'attivazione è quasi lineare.

- **Covariate shift**: l'input di un layer profondo dipende da tutti i pesi precedenti e cambia molto durante il training. BN ne mantiene fisse media e varianza.
- Media e varianza del mini-batch aggiungono un piccolo rumore, che fa da **regolarizzazione** (più forte con batch piccoli).
- La normalizzazione elimina il bias: con BN non servono bias unit.
- **A test time** si usano le medie mobili esponenziali di μ_i e σ_i^2 calcolate durante il training.

Nel detector ogni blocco è Conv (senza bias) $\rightarrow$ BN $\rightarrow$ ReLU.

Dalla teoria ai due esercizi

Quello che serve a entrambi

Rete profonda con attivazioni **ReLU**, loss di **cross-entropy**, backpropagation, mini-batch con **Adam**, normalizzazione, regolarizzazione e scelta del modello sul **validation**.

Object detection

Immagini → rete **convoluzionale**, che sfrutta la struttura spaziale dei pixel. *Architettura YOLO*.

Sentiment analysis

Frase → **Transformer**, che mette in relazione ogni parola con tutte le altre. *Modello BERT*.

In entrambi si parte da una rete **pre-addestrata** (*transfer learning*).

- 1 Introduzione al deep learning
- 2 Addestrare una rete profonda
- 3 Object Detection (YOLO)**
- 4 Sentiment Analysis (Transformer)
- 5 Conclusioni

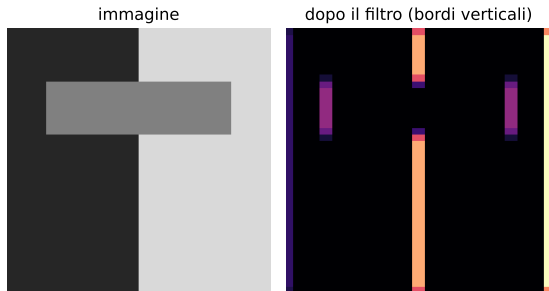
Reti convoluzionali: la convoluzione

Una rete densa su immagini grandi è **ingestibile** (troppi pesi).

La **convoluzione** fa scorrere un piccolo **filtro** (kernel) sull'immagine, calcolando un prodotto scalare a ogni posizione → produce una *feature map*.

Filtri appresi, non progettati

Un filtro può rilevare i **bordi** (verticali, orizzontali). Invece di costruirlo a mano (Sobel, Scharr), si **apprendono i pesi del filtro** con la backpropagation.



Un filtro verticale evidenzia i bordi verticali.

CNN: padding, stride, pooling

Iperparametri della convoluzione

- **Padding**: bordo di zeri; *same* mantiene la dimensione, *valid* la riduce.
- **Stride**: di quanti passi scorre il filtro.
- **Più filtri** → più feature map (profondità). Su immagini RGB: convoluzione su ogni canale, poi somma.

Pooling

Il **max pooling** tiene il valore massimo di ogni regione: riduce la dimensione e agisce da rilevatore di feature.

Architettura tipica:

CONV → POOL → ... → FC

Andando in profondità: più filtri, dimensioni spaziali più piccole.

Perché la convoluzione funziona

Due proprietà chiave

- **Parameter sharing**: un rilevatore di feature (es. un bordo) è utile in **tutta** l'immagine → pochissimi parametri, che *non dipendono* dalla dimensione dell'immagine.
- **Sparsità delle connessioni**: ogni output dipende solo da una **piccola regione** dell'input → meno parametri, meno overfitting.

Nel detector: la ResNet riduce 416×416 fino alla griglia 26×26 (stride 16); il “collo” aggiunge due blocchi Conv → BN → ReLU a risoluzione costante.

Il problema: localizzazione con bounding box

Object detection

Data un'immagine, individuare **quali** oggetti contiene e **dove** si trovano, disegnando una *bounding box* attorno a ciascuno.

Dataset: Object365 (365 classi, 600k immagini, 10M box).

- Volume incompatibile con un PC $\Rightarrow$ **subsampling** su due assi.
- **Immagini**: 15 shard scaricati $\rightarrow$ **14.361** immagini (119.215 box), divise in train 11.467 / validation 1.395 / test 1.499.
- **Classi**: da 365 a **8**, scelte *misurando* frequenza e dimensione: Person, Chair, Table, Cabinet, Car, Lamp, Picture, Monitor.
- Categorie “sorelle” **unite** (Car + SUV + Van + Pickup): altrimenti un SUV sarebbe sfondo accanto a un'auto identica etichettata come oggetto.

Tecnica scelta: **YOLO**. Griglia, anchor, loss, IoU e NMS sono *implementati a mano* (nessuna libreria di detection).

Dalla sliding window a YOLO

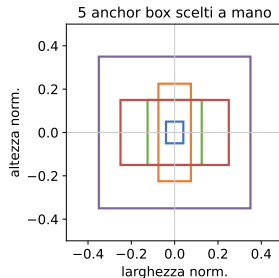
Sliding window: si fa scorrere una finestra e si classifica ogni ritaglio.

- Problema principale: **costo computazionale**.
- Soluzione: implementarla **convoluzionalmente**.

YOLO: griglia $S \times S$; ogni oggetto assegnato a **una cella** (quella del suo centro), che ne predice box e classe.

Anchor box

Per ogni cella, n forme predefinite. Se più oggetti cadono nella stessa cella, ognuno va all'anchor più simile per forma (IoU).

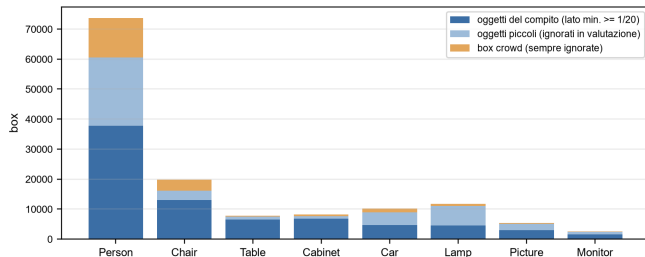


Distribuzione del dataset

Sottoinsieme **sbilanciato**: **Person** è il 51% delle box, **Monitor** meno del 2%
⇒ prestazioni attese **inferiori sulle classi rare**.

Regioni ignorate (*don't care*)

Box **crowd** (una box su un gruppo di oggetti, il 15% delle annotazioni) e oggetti **più piccoli di una cella**: etichetta "?", niente obiettivi né penalità. Cancellarli insegnerebbe che lì *non c'è nulla*.



Backbone convoluzionale con downsampling $\times 32$, seguito da una **testa di detection** 1×1 scritta da noi.

Input $3 \times 416 \times 416$

ResNet34 troncata a layer3 (stride 16)
→ $256 \times 26 \times 26$ (griglia S)

Collo: $2 \times (\text{conv } 3 \times 3 + \text{BN} + \text{ReLU})$

Testa $1 \times 1 \rightarrow A \times (5+C) = 65$ canali

Output (5, 26, 26, 13)

- Rete **completamente convoluzionale**.
- **3.380** box candidate per immagine ($5 \times 26 \times 26$).

Backbone: transfer learning

ResNet34 pre-addestrata su ImageNet e fine-tunata. Alternative provate: ResNet18 ($-0,05$ di mAP) e un backbone scritto interamente da zero.

Anchor, target, loss, IoU e NMS restano **implementati a mano**: cambia solo l'estrattore di feature. Il **troncamento** decide il compromesso: stride piccolo → griglia fine ma campo recettivo ridotto; stride grande → vede più contesto ma celle grossolane.

La loss YOLO

La rete predice *logit grezzi*, decodificati come (YOLOv2):

$$b_x = \sigma(t_x) + c_x, \quad b_y = \sigma(t_y) + c_y, \quad b_w = a_w e^{t_w}, \quad b_h = a_h e^{t_h}$$

Quattro componenti

Componente	Funzione	Peso	Si applica a
Coordinate	MSE	$\lambda = 5$	anchor responsabili
Objectness	BCE	1	anchor responsabili
No-object	BCE	$\lambda = 10$	anchor vuoti <i>e non ignorati</i>
Classe	Cross-entropy	1	anchor responsabili

Su 3.380 box quasi tutte sono vuote: il termine no-object è normalizzato come **media sulle celle vuote** (non dipende dalla griglia) e pesato $\lambda = 10$, il rapporto del YOLO originale. Troppo basso $\rightarrow$ oggetti ovunque; troppo alto $\rightarrow$ “non c'è mai niente”.

Inferenza: non-max suppression

Ogni box: **score** = objectness $\times$ prob. classe.

- 1 Scarta le box sotto soglia (0,25).
- 2 **NMS** per classe.

Non-max suppression

ordina le box per score

finché restano box:

prendi quella con score max $\rightarrow$ predizione

scarta le altre con IoU > soglia

NMS *per classe*: due oggetti di classi diverse possono sovrapporsi (una persona con un cappello); due box sovrapposte della stessa classe no.

Come si misura un detector

Una predizione è **corretta** se ha $\text{IoU} \geq 0,5$ con un oggetto della stessa classe non ancora abbinato; le altre sono falsi positivi.

$$P = \frac{TP}{TP + FP} \quad R = \frac{TP}{N_{\text{ogg}}} \quad F_1 = \frac{2PR}{P + R}$$

La trappola della soglia fissa

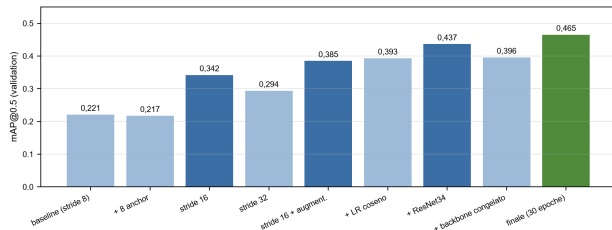
Lo stesso modello: **F1 0,145** a soglia 0,25 e **0,249** a 0,65. Confrontare modelli a soglia fissa è ingannevole.

mAP@0.5

Si fa variare la soglia, si costruisce la curva **precision-recall** e se ne prende l'**area (AP)**, mediata sulle classi. **Non dipende dalla soglia** $\Rightarrow$ è la metrica con cui si confrontano le varianti e si sceglie il checkpoint.

La soglia d'uso (0,9) è poi quella che massimizza l'**F1** sul validation, salvata nel checkpoint.

Ottimizzazione e risultati



Un esperimento per modifica: così ogni guadagno ha una causa.

stride 16 +**0,121** (campo recettivo ~ 275 px invece di 131)
· **augmentation** +**0,043** (via l'overfitting) · **ResNet34**
+**0,052**

Scartati: 8 anchor ($-0,004$: coprono meglio le forme, ma non erano il collo di bottiglia) e backbone congelato.

Modello finale sul **test set**, valutato **una sola volta**:

mAP@0.5	0,444
F1	0,588
Precision / Recall	0,660 / 0,531

Baseline 0,221 $\Rightarrow$ **il doppio**. Per classe: Person 0,72, Table 0,19 (tavoli coperti e tagliati).

Recall oggetti **grandi** 0,58 > **medi** 0,51: il problema del campo recettivo è risolto (nella baseline 0,38 contro 0,56).

Demo dal vivo nel notebook: detection su una foto scattata con il telefono, mai vista dal modello.

- 1 Introduzione al deep learning
- 2 Addestrare una rete profonda
- 3 Object Detection (YOLO)
- 4 Sentiment Analysis (Transformer)**
- 5 Conclusioni

Il problema: classificare frasi

Sentiment analysis

Classificare una sequenza testuale come **positiva** o **negativa**, usando un'architettura **transformer**.

- **Dataset:** SST-2 (frasi brevi etichettate pos/neg, da HuggingFace).
- Corrisponde alla traccia (“frasi positive o negative”); nessun token Kaggle richiesto.
- Tre split: **train**; **validation** (2000 frasi ricavate dal train) per scegliere il checkpoint; **test** (le 872 frasi del validation ufficiale) usato *una sola volta* per i risultati.

Il codice supporta comunque un CSV Kaggle via `--dataset csv`.

Le **RNN** codificano bene la storia recente, ma faticano su contesti lontani e sono **sequenziali** (lente).

Esempio delle slide

"I grew up in France, ... thus I speak ??" — la parola da predire (*French*) dipende da un termine molto **lontano** nella sequenza.

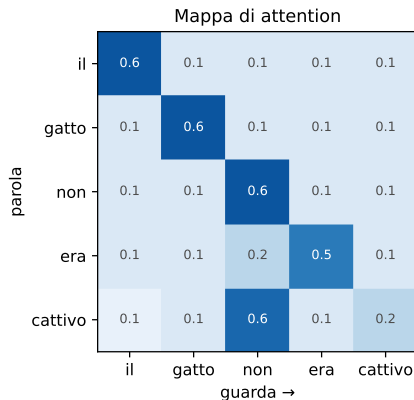
La risposta è il **meccanismo di attention**: per produrre la rappresentazione di una parola, la si mette in relazione con **tutte** le altre e si pesa quali sono rilevanti, indipendentemente dalla distanza.

Self-attention

L'output per ogni parola è una **somma pesata** delle rappresentazioni di tutte le parole; i pesi misurano quanto ognuna è *rilevante*.

Proprietà (dalle slide)

- Calcolo **parallelo** di tutti gli output (non sequenziale).
- *Permutation equivariant*: è più un modello di **insiemi** che di sequenze $\Rightarrow$ serve il **positional encoding** per reintrodurre l'ordine.



cattivo mette attenzione su *non*.

Query, Key, Value e multi-head

Tre ruoli, tre proiezioni lineari apprese

- **Query**: il vettore che viene confrontato con tutti gli altri.
- **Key**: il vettore contro cui la query è confrontata.
- **Value**: ciò che si somma (pesato) per formare l'output.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) V$$

- Divisione per $\sqrt{d}$: **scaled** attention (stabilità).
- **Multi-head**: più “teste” in parallelo, ciascuna cattura un tipo diverso di relazione tra le parole.

Il blocco Transformer

Un blocco impila due sotto-strati

- 1 **Multi-head self-attention**
- 2 **Rete feed-forward** (applicata a ogni posizione)

Ciascuno preceduto da una **layer normalization** e seguito da una **connessione residua** (skip, *Pre-LN*):

$$x + \text{Sublayer}(\text{LayerNorm}(x))$$

Impilando più blocchi si ottiene il Transformer.

Positional encoding

La self-attention è *permutation equivariant*: si **somma** a ogni token un vettore che ne codifica la **posizione**, così il modello conosce l'ordine.

Layer normalization

Come la batch norm, ma normalizza sulle **feature di ogni token** (non sul batch).

Il Transformer e BERT

Transformer

Un modello che usa principalmente la **self-attention** per propagare l'informazione tra le unità (token di testo, ma anche pixel: Vision Transformer).

BERT: transformer **encoder-only**

bidirezionale, pre-addestrato con il **masking** (predire token nascosti). Adatto a *comprendere* una frase, non a generarla.

Il nostro uso

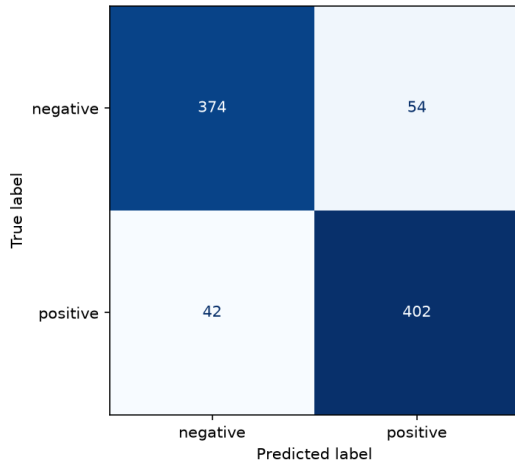
Fine-tuning (transfer learning): si parte da BERT pre-addestrato e si aggiunge una testa di classificazione binaria. Modello: **DistilBERT**, un BERT più leggero (6 strati invece di 12).

Risultati

Fine-tuning su GPU ($\sim 1,5$ min). Metriche sul **test set** (mai usato per addestrare né per scegliere il modello):

Metrica	Valore
Accuracy	89,0%
F1	0,893
Precision	0,882
Recall	0,905

Errori: 54 negative $\rightarrow$ positive, 42 positive $\rightarrow$ negative. Leggera tendenza verso *positive*: il 56% del train è positivo.



Analisi qualitativa: il caso della negazione

Frase costruite ad hoc:

Output del modello

```
[positive 99.9%] I absolutely loved this film
[negative 99.6%] the worst purchase I have ever made
[positive 99.8%] It is not bad at all, quite enjoyable
[negative 83.2%] the acting was fine, the story less so
```

- “*not bad*”: contiene *bad* ma è classificata **positiva** al 99,8%. L'attention collega *bad* a *not* — è l'**esempio della slide 11**.
- Frase a sentimento misto $\Rightarrow$ confidenza più bassa (83%): il modello *segnala* l'incertezza.

- 1 Introduzione al deep learning
- 2 Addestrare una rete profonda
- 3 Object Detection (YOLO)
- 4 Sentiment Analysis (Transformer)
- 5 Conclusioni**

	Esercizio 1 — Detection	Esercizio 2 — Sentiment
Tecnica	YOLO implementato a mano	DistilBERT fine-tunato
Dati	14.361 immagini, 8 classi	SST-2, 20.000 frasi
Metodo	train / validation / test separati	train / validation / test separati
Ottimizzazione	8 esperimenti controllati	—
Risultati (test)	mAP@0.5 0,444 (da 0,221)	Acc. 89,0% — F1 0,893

- **Es. 1:** il detector funziona su foto reali; l'ottimizzazione ha **raddoppiato** il mAP, con ogni scelta motivata da una misura.
- **Es. 2:** risultati solidi, allineati a DistilBERT su SST-2.
- In entrambi il **test set** è stato usato una sola volta, a scelte concluse.

Aderenza al programma del corso

Nucleo del progetto: tecniche viste a lezione. YOLO con griglia, anchor scelti a mano e assegnati per IoU, NMS; etichette *don't care* (le nostre **regioni ignorate**); CNN, ResNet34, batch norm; **transfer learning** (backbone pre-addestrato, fine-tuning di BERT); transformer e self-attention; Adam, regolarizzazione L2, early stopping; data augmentation.

Aggiunto oltre il programma, con motivazione

- **mAP@0.5** (Es. 1): precision e recall dipendono dalla soglia, quindi non confrontano modelli calibrati diversamente.
- **Learning rate variabile**: decadimento coseno (Es. 1, +0,008 di mAP, misurato); warmup e decadimento lineare (Es. 2, standard per il fine-tuning di BERT: evita passi grandi sui pesi pre-addestrati).
- **AdamW** (Es. 2): Adam con il weight decay applicato separatamente dal gradiente, ottimizzatore predefinito del Trainer HuggingFace.
- **DistilBERT** e **fp16** (Es. 2): un BERT più piccolo ottenuto per distillazione e calcoli a mezza precisione, solo per ridurre tempo e memoria.

Le aggiunte in Es. 1 sono state **misurate** con esperimenti controllati.

Grazie per l'attenzione

Matteo Barbieri — Progetto di Apprendimento Automatico e Profondo
Object Detection & Sentiment Analysis